

POSTGRESQL

VACUUM, Autovacuum & Bloat Management

Technical SOP & Best-Practice Guide

MVCC • Dead Tuples • Visibility Map • XID Freeze • Bloat • HOT • Troubleshooting

Document Type	Production DBA SOP
Scope	PostgreSQL VACUUM, autovacuum, ANALYZE, bloat, wraparound and monitoring
Audience	PostgreSQL DBAs, SREs, Platform Engineers and Technical Architects
Primary Authority	PostgreSQL Official Documentation

1. Purpose and Scope

This SOP provides a production-focused method for understanding, monitoring, tuning and troubleshooting PostgreSQL VACUUM, autovacuum, dead tuples, table/index bloat, planner statistics and transaction ID wraparound protection. It is designed for real operational use rather than as a generic configuration recipe.

- Keep dead-tuple accumulation under control.
- Make reusable space available quickly.
- Maintain the visibility map to support efficient index-only scans.
- Keep optimizer statistics current through ANALYZE.
- Prevent transaction ID and MultiXact wraparound.
- Reduce avoidable heap and index growth.
- Keep autovacuum ahead of sustained write workload.
- Avoid unnecessary VACUUM FULL operations.

2. MVCC and Why VACUUM Exists

PostgreSQL uses Multi-Version Concurrency Control (MVCC). An UPDATE normally creates a new tuple version rather than overwriting the old row in place. A DELETE marks the old tuple version as no longer visible to future transactions, but its physical storage is not immediately removed.

```
Original row
|
| UPDATE
v
Old Tuple Version ---> New Tuple Version
(becomes dead when no snapshot needs it)
```

Once an old tuple version is no longer visible to any transaction, VACUUM can reclaim its storage for reuse. Dead tuples are therefore a normal result of MVCC; the operational problem is when they are generated faster than vacuum can clean them.

3. Dead Tuples and Basic Monitoring

A dead tuple is an obsolete row version that is no longer required by active snapshots. UPDATE and DELETE workloads can therefore generate significant dead-tuple volume.

```
SELECT
  schemaname, relname,
  n_live_tup, n_dead_tup,
  n_tup_ins,
  n_tup_upd,
  n_tup_del,
  n_tup_hot_upd,
  last_vacuum,
  last_autovacuum,
  vacuum_count,
  autovacuum_count
FROM pg_stat_user_tables
ORDER BY n_dead_tup DESC;
```

Important: `n_live_tup` and `n_dead_tup` are statistical estimates. Use them for trend analysis and prioritization, not as exact physical tuple counts.

4. What VACUUM Actually Does

- Identifies tuple versions that are no longer needed.
- Makes dead-tuple space reusable.
- Cleans dead index entries when appropriate.
- Updates the Free Space Map (FSM).
- Maintains the Visibility Map (VM).
- Freezes sufficiently old transaction IDs.
- Advances maintenance metadata used by PostgreSQL.

```
VACUUM public.orders;

VACUUM (VERBOSE) public.orders;

VACUUM (ANALYZE) public.orders;
```

For routine operations, standard `VACUUM` should normally be preferred over `VACUUM FULL`.

5. VACUUM Does Not Normally Shrink the Data File

A normal `VACUUM` primarily makes internal space reusable. It does not generally rewrite the relation or return most free space to the operating system. PostgreSQL can sometimes truncate completely empty pages at the physical end of a relation, but that is not equivalent to `VACUUM FULL`.

```
Before VACUUM
[Live][Dead][Live][Dead][Dead][Live]

After VACUUM
[Live][Free][Live][Free][Free][Live]
```

Operation	Primary Outcome
<code>VACUUM</code>	Makes internal space reusable and performs routine maintenance
<code>VACUUM FULL</code>	Rewrites the table and can return substantial space to the OS

6. Free Space Map and Visibility Map

Free Space Map (FSM)

The FSM tracks relation pages that contain free space. PostgreSQL can reuse that space for future `INSERT` or `UPDATE` activity, which is why a large relation can still be healthy after `VACUUM`.

Visibility Map (VM)

The visibility map tracks pages known to be all-visible and, separately, all-frozen. This allows `VACUUM` to skip work on some pages and helps PostgreSQL perform index-only scans without visiting the heap for every visibility check.

7. VACUUM vs VACUUM FULL

Characteristic	Standard VACUUM	VACUUM FULL
Routine maintenance	Yes	No
Reuses dead-tuple space	Yes	Yes
Rewrites entire table	No	Yes
Returns significant space to OS	Usually no	Yes
Normal DML access	Generally available	Blocked by ACCESS EXCLUSIVE lock
Extra disk required	Limited	Significant
Production risk	Lower	Higher
Recommended routinely	Yes	No

```
VACUUM (FULL, ANALYZE) public.orders;
```

Use VACUUM FULL only when the operational benefit justifies blocking, rewrite duration, WAL generation, replication impact and additional disk requirements.

8. When VACUUM FULL May Be Appropriate

- A very large percentage of a table was permanently deleted.
- The table is unlikely to reuse the free space naturally.
- Storage must be returned to the operating system.
- A maintenance window is available.
- The application can tolerate the required lock.
- Sufficient temporary/free disk space exists.

A recurring schedule such as “VACUUM FULL every Sunday” is generally a warning sign that the underlying vacuum/bloat strategy needs improvement.

9. Autovacuum Architecture

```
SHOW autovacuum;
```

Production systems should generally keep autovacuum enabled. PostgreSQL uses an autovacuum launcher and workers to automate VACUUM and ANALYZE work. Anti-wraparound vacuuming is a safety mechanism and should not be casually disabled or interrupted.

```
SELECT name, setting, unit, context, source, pending_restart
FROM pg_settings
WHERE name LIKE 'autovacuum%'
ORDER BY name;
```

10. Autovacuum Trigger Logic

For modern PostgreSQL releases, the vacuum trigger is based on a fixed threshold plus a fraction of estimated table tuples, subject to version-specific parameters such as `autovacuum_vacuum_max_threshold` where available.

```
vacuum threshold =
min(
    autovacuum_vacuum_max_threshold,
    autovacuum_vacuum_threshold
    + autovacuum_vacuum_scale_factor × estimated table tuples
)
```

With a 10,000,000-row table, a threshold of 50 and a scale factor of 0.20 produces a proportional trigger near 2,000,050 changes before considering any maximum threshold. This illustrates why large, high-write tables often need per-table tuning.

11. Per-Table Autovacuum Tuning

For large or high-write tables, per-table storage parameters are usually safer than globally aggressive settings.

```
ALTER TABLE public.orders SET (
    autovacuum_vacuum_scale_factor = 0.01,
    autovacuum_vacuum_threshold = 10000,
    autovacuum_analyze_scale_factor = 0.01
);
SELECT
    n.nspname AS schema_name,
    c.relname AS table_name,
    c.reloptions
FROM pg_class c
JOIN pg_namespace n ON n.oid = c.relnamespace
WHERE c.reloptions IS NOT NULL
ORDER BY 1, 2;
```

Treat values such as 0.01 or 10,000 only as examples. Production values must be derived from table size, write rate, storage capacity, vacuum duration and acceptable maintenance overhead.

12. Autovacuum Cost Control and Memory

Vacuum cost-based throttling limits maintenance pressure on storage. Important settings include `autovacuum_vacuum_cost_delay`, `autovacuum_vacuum_cost_limit` and the `vacuum_cost_page_*` parameters.

Overly conservative cost settings can make autovacuum too slow to catch up. Tune against observed disk latency, IOPS, throughput, WAL volume, checkpoint behavior, replication lag, worker saturation and application latency.

```
SHOW autovacuum_work_mem;
```

Memory planning must consider memory per worker multiplied by concurrent workers. Do not set very large `autovacuum_work_mem` values without considering total server memory exposure.

13. Detecting Vacuum Backlog

```
SELECT
    schemaname,
    relname,
    n_live_tup,
    n_dead_tup,
    ROUND(100.0 * n_dead_tup / NULLIF(n_live_tup + n_dead_tup, 0), 2) AS dead_tuple_pct,
    last_vacuum,
    last_autovacuum,
    vacuum_count,
    autovacuum_count
FROM pg_stat_user_tables
WHERE n_dead_tup > 0
ORDER BY n_dead_tup DESC;
```

Do not label a table bloated from `n_dead_tup` alone. Compare dead-tuple count, percentage, table size, update/delete rate and trend over time.

14. Monitoring Active Autovacuum and VACUUM Progress

```
SELECT
    pid, datname, username, state, wait_event_type, wait_event,
    query_start, clock_timestamp() - query_start AS runtime, query
FROM pg_stat_activity
WHERE backend_type = 'autovacuum worker'
ORDER BY query_start;

SELECT
    pid,
    datname,
    relid::regclass AS relation,
    phase,
    heap_blks_total,
    heap_blks_scanned,
    heap_blks_vacuumed,
    index_vacuum_count,
    num_dead_item_ids,
    max_dead_item_ids
FROM pg_stat_progress_vacuum;
```

Column availability can differ by PostgreSQL version. Validate monitoring queries against the version you operate.

15. Long-Running Transactions and Idle-in-Transaction Sessions

Old snapshots can prevent VACUUM from removing dead tuples that might still be visible to a transaction. Long-running and idle-in-transaction sessions are therefore common root causes of cleanup delay.

```

SELECT
    pid, username, datname, state, xact_start,
    clock_timestamp() - xact_start AS transaction_age,
    backend_xmin, query
FROM pg_stat_activity
WHERE xact_start IS NOT NULL
ORDER BY xact_start;

SELECT
    pid, username, datname, client_addr, xact_start, state_change,
    clock_timestamp() - xact_start AS transaction_age, query
FROM pg_stat_activity
WHERE state = 'idle in transaction'
ORDER BY xact_start;

```

Where appropriate, `idle_in_transaction_session_timeout` can help protect the database from abandoned transactions, but only after confirming application behavior.

16. Prepared Transactions and Replication Slots

```

SELECT transaction, gid, prepared, owner, database
FROM pg_prepared_xacts
ORDER BY prepared;

SELECT
    slot_name, slot_type, active, active_pid, xmin, catalog_xmin,
    restart_lsn, confirmed_flush_lsn
FROM pg_replication_slots;

```

Important distinction: old `xmin` can retain heap tuple visibility requirements; `catalog_xmin` can retain catalog tuples; old `restart_lsn` primarily causes WAL retention. Do not assume every inactive slot directly causes heap bloat.

17. Transaction ID Wraparound Protection

PostgreSQL transaction IDs are finite, so old tuple XIDs must eventually be frozen. VACUUM is responsible for this maintenance. Transaction ID age should be monitored proactively instead of waiting for anti-wraparound emergency behavior.

```

SELECT datname, age(datfrozenxid) AS xid_age
FROM pg_database
ORDER BY age(datfrozenxid) DESC;

SELECT
    n.nspname AS schema_name,
    c.relname,
    age(c.relfrozenxid) AS xid_age
FROM pg_class c
JOIN pg_namespace n ON n.oid = c.relnamespace
WHERE c.relkind IN ('r', 'm')
ORDER BY age(c.relfrozenxid) DESC
LIMIT 50;

SHOW autovacuum_freeze_max_age;
SHOW vacuum_failsafe_age;

```

Do not interrupt anti-wraparound vacuum casually. Wraparound risk can become a database availability incident.

18. Measuring Physical Table Bloat

`pg_stat_user_tables` is useful for triage, but `n_dead_tup` is not an exact physical bloat measurement. The `pgstattuple` extension provides deeper inspection.

```
CREATE EXTENSION IF NOT EXISTS pgstattuple;

SELECT *
FROM pgstattuple('public.orders');

SELECT *
FROM pgstattuple_approx('public.orders');
```

Exact inspection can be expensive on very large tables. Use monitoring first, then deeper physical inspection only where justified. `pgstattuple_approx()` is often a lower-impact first choice.

19. Table Bloat vs Index Bloat

Heap bloat and index bloat are related but different. A table can have acceptable heap utilization while one or more indexes are inefficient, and vice versa. Diagnose them separately.

```
REINDEX INDEX index_name;
REINDEX TABLE table_name;
REINDEX INDEX CONCURRENTLY index_name;
```

Do not automatically rebuild every PostgreSQL index on a fixed schedule. Reindex after evidence shows that the index needs intervention.

20. HOT Updates and Fillfactor

Heap-Only Tuple (HOT) updates can avoid creating new index entries when PostgreSQL can keep the new tuple version on the same heap page and no indexed column requires a new index entry. This can reduce index churn significantly.

```
SELECT
    schemaname, relname, n_tup_upd, n_tup_hot_upd,
    ROUND(100.0 * n_tup_hot_upd / NULLIF(n_tup_upd, 0), 2) AS hot_update_pct
FROM pg_stat_user_tables
WHERE n_tup_upd > 0
ORDER BY n_tup_upd DESC;

ALTER TABLE public.orders SET (fillfactor = 80);
```

Lower fillfactor leaves more free page space and can increase HOT opportunities, but it also increases baseline table size. Use only when the workload justifies it.

21. ANALYZE and Planner Statistics

VACUUM and ANALYZE solve different problems. VACUUM handles dead-tuple cleanup, reusable space, visibility map maintenance and freezing. ANALYZE collects statistics used by the optimizer.

```
ANALYZE public.orders;

VACUUM (ANALYZE) public.orders;
```


22. Partitioned and Temporary Tables

Autovacuum processes physical partitions because they hold tuples. The partitioned parent does not contain those tuples, so parent-level statistics may require explicit ANALYZE after bulk loads, partition attachment or major distribution changes.

```
ANALYZE public.sales_history;
```

Autovacuum cannot maintain another session's temporary tables. Long-running sessions with large temporary tables may require explicit VACUUM or ANALYZE when appropriate.

23. Logging Autovacuum for Investigation

```
ALTER SYSTEM SET log_autovacuum_min_duration = '2s';
SELECT pg_reload_conf();
```

Use autovacuum logging during controlled diagnostic periods to review duration, tuples removed, pages scanned, pages skipped, index scans, buffer activity, WAL generation and tuples that could not yet be removed. Avoid excessive logging indefinitely.

24. Baseline Before Any Tuning Change

- Database size and relation sizes.
- n_live_tup and n_dead_tup trends.
- last_autovacuum, last_autoanalyze and counters.
- DML rates: inserts, updates, deletes and HOT updates.
- XID age and freeze risk.
- Active autovacuum workers and vacuum duration.
- Long-running transactions and idle-in-transaction sessions.
- Prepared transactions and replication slot horizons.
- Storage latency, IOPS and throughput.
- WAL generation and replication lag.

```
SELECT
    name, setting, unit, source, context, pending_restart
FROM pg_settings
WHERE name LIKE 'autovacuum%'
    OR name LIKE 'vacuum%'
ORDER BY name;
```

25. Recommended Production Change Process

1. Establish a baseline.
2. Identify the problematic relations.
3. Determine why vacuum is behind.
4. Check blockers and retained horizons.
5. Check worker capacity and storage headroom.
6. Tune one or a small set of tables first.
7. Monitor for at least one representative workload cycle.

8. Compare against baseline.
9. Expand carefully only after proving benefit.

26. Troubleshooting Scenarios

Scenario A — Autovacuum Never Starts on a Large Table

- Check whether thresholds are actually reached.
- Check whether autovacuum is disabled at relation level.
- Review reloptions for the table.
- Check worker pressure and global settings.
- Confirm statistics are being updated as expected.

```
SELECT reloptions
FROM pg_class
WHERE oid = 'public.orders'::regclass;

SHOW autovacuum;
SHOW autovacuum_vacuum_threshold;
SHOW autovacuum_vacuum_scale_factor;
```

Scenario B — Autovacuum Runs but Bloat Keeps Growing

- Compare dead-tuple generation rate with cleanup capacity.
- Check whether cost throttling is too conservative.
- Check whether workers are saturated.
- Check long-running transactions and old xmin horizons.
- Check prepared transactions and replication slots.
- Check table size, index count and storage latency.

Scenario C — Massive DELETE Completed

A normal VACUUM makes dead space reusable, but the physical relation may remain large. If the table will grow again, retaining reusable space may be desirable. If storage must actually be reclaimed and the application can tolerate the lock and rewrite, consider a controlled rewrite strategy such as VACUUM FULL.

Scenario D — Bulk Load Completed

After a major bulk load, run ANALYZE where needed so planner statistics reflect the new data distribution before critical production traffic resumes.

27. Rollback of Tuning Changes

```
ALTER TABLE public.orders RESET (
    autovacuum_vacuum_scale_factor,
    autovacuum_vacuum_threshold,
    autovacuum_analyze_scale_factor,
    autovacuum_analyze_threshold
);
```

```
ALTER SYSTEM RESET autovacuum_vacuum_cost_limit;
ALTER SYSTEM RESET autovacuum_vacuum_cost_delay;
SELECT pg_reload_conf();
```

After rollback, verify the active value, source and pending_restart status in pg_settings.

28. Production Monitoring Query

```
SELECT
    schemaname,
    relname,
    n_live_tup,
    n_dead_tup,
    ROUND(100.0 * n_dead_tup / NULLIF(n_live_tup + n_dead_tup, 0), 2) AS dead_pct,
    n_tup_ins,
    n_tup_upd,
    n_tup_del,
    n_tup_hot_upd,
    last_vacuum,
    last_autovacuum,
    last_analyze,
    last_autoanalyze,
    vacuum_count,
    autovacuum_count,
    analyze_count,
    autoanalyze_count
FROM pg_stat_user_tables
ORDER BY n_dead_tup DESC;
```

29. Daily and Weekly DBA Checklist

Daily Checklist

- ☐ Autovacuum enabled.
- ☐ High dead-tuple relations reviewed.
- ☐ Large relations with old last_autovacuum investigated.
- ☐ Long-running transactions reviewed.
- ☐ Idle-in-transaction sessions reviewed.
- ☐ Old prepared transactions checked.
- ☐ Replication slots and retained horizons checked.
- ☐ Active autovacuum workers and VACUUM progress checked.
- ☐ XID age reviewed.
- ☐ Storage latency and replication lag monitored.
- ☐ Unexpected table/index growth investigated.

Weekly Trend Review

- ☐ Dead-tuple generation rate.
- ☐ Autovacuum frequency and duration.
- ☐ Table and index growth.
- ☐ Storage latency, IOPS and throughput.

- ☐ XID age progression.
- ☐ VACUUM backlog and recurring blockers.

30. Production Best Practices

- Keep autovacuum enabled.
- Prefer per-table tuning for very large/high-write relations.
- Favor frequent, manageable cleanup over rare massive cleanup.
- Keep transactions short.
- Control idle-in-transaction behavior.
- Monitor replication xmin/catalog_xmin and prepared transactions.
- Monitor transaction age early.
- Avoid routine VACUUM FULL.
- Measure before and after tuning.
- Consider storage capacity before making autovacuum more aggressive.
- Use ANALYZE appropriately after significant data distribution changes.
- Use HOT/relfillfactor tuning only when measurements support it.

31. Common DBA Mistakes

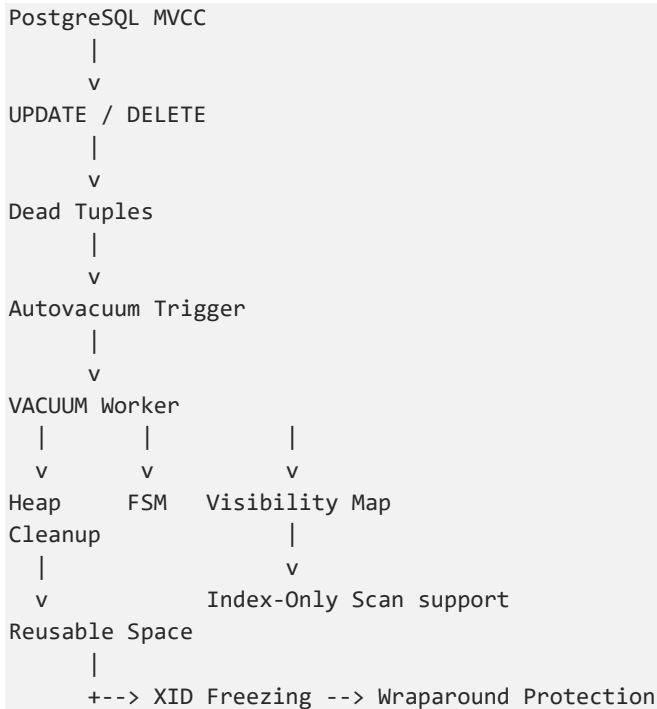
- Avoid: Disabling autovacuum globally.
- Avoid: Running VACUUM FULL on every table weekly.
- Avoid: Treating n_dead_tup as exact physical bloat.
- Avoid: Using one scale factor for every table regardless of size/workload.
- Avoid: Increasing workers without checking storage capacity.
- Avoid: Increasing autovacuum_work_mem without total memory planning.
- Avoid: Ignoring long-running or idle-in-transaction sessions.
- Avoid: Ignoring prepared transactions or replication horizons.
- Avoid: Ignoring XID age.
- Avoid: Assuming VACUUM always reduces physical file size.
- Avoid: Reindexing every index without evidence.
- Avoid: Copying blog tuning values directly into production.

32. Recommended Troubleshooting Order

10. Identify affected relations.
11. Measure dead-tuple trend and write rate.
12. Measure relation growth.
13. Check last autovacuum/autoanalyze.
14. Check active workers and pg_stat_progress_vacuum.
15. Check long-running and idle-in-transaction sessions.
16. Check prepared transactions.
17. Check replication xmin/catalog_xmin.
18. Check per-table and global thresholds.

19. Check worker capacity and cost throttling.
20. Check storage latency/throughput.
21. Check XID and MultiXact age.
22. Measure physical bloat if required.
23. Tune selected relations.
24. Monitor results.
25. Escalate to rewrite/reindex only when justified.

33. Architecture Summary



34. Final Operational Guidance

PostgreSQL bloat should not be managed by periodically running VACUUM FULL. A mature strategy monitors workload, keeps transactions short, tunes autovacuum with evidence, uses per-table settings where needed, protects the transaction horizon and reserves rewrite/reindex operations for cases that are actually justified.

The desired steady state is not zero dead tuples. On an active MVCC database, dead tuples are normal. The healthy state is that dead tuples are generated and cleaned at a sustainable rate, reusable space is recovered, statistics remain useful, visibility information stays current, old transaction IDs are safely frozen and relation growth remains controlled without harming application performance.

35. Reference Documentation

PostgreSQL Routine Vacuuming: <https://www.postgresql.org/docs/current/routine-vacuuming.html>

VACUUM Command: <https://www.postgresql.org/docs/current/sql-vacuum.html>

ANALYZE Command: <https://www.postgresql.org/docs/current/sql-analyze.html>

Automatic Vacuuming Configuration: <https://www.postgresql.org/docs/current/runtime-config-autovacuum.html>

Vacuum Cost-Based Delay: <https://www.postgresql.org/docs/current/runtime-config-resource.html#RUNTIME-CONFIG-RESOURCE-VACUUM-COST>

VACUUM Progress Reporting: <https://www.postgresql.org/docs/current/progress-reporting.html#VACUUM-PROGRESS-REPORTING>

pgstattuple: <https://www.postgresql.org/docs/current/pgstattuple.html>

HOT Updates: <https://www.postgresql.org/docs/current/storage-hot.html>

Operational rule: PostgreSQL parameter availability, defaults and behavior can change by major version. Always validate this SOP against the official documentation for the exact PostgreSQL version running in production before applying configuration changes.